

Operating Systems Preparation Handbook

Threads, synchronisation, system calls, memory, scheduling —
and how they meet in SWEB

Companion to the twelve modules in `preparations/`

Every chapter belongs to the module with the same number: read the chapter, run the module's examples, then do its assignment. The boxes titled *In SWEB* point to the place in the course's kernel where the idea turns into code you will change for A1.

Contents

0	The C toolbox	4
0.1	Toolchain: from source to a running program	4
0.2	Sanitizers: let the compiler find your bugs	4
0.3	Pointers	5
0.4	Lifetimes: stack, heap, static	6
0.5	Function pointers and callbacks	6
1	Processes	8
1.1	What the kernel keeps per process	8
1.2	The address space	8
1.3	fork, exec, wait, exit	9
1.4	File descriptors and pipes	10
1.5	Signals	10
1.6	A trap: buffered output and fork	10
2	Threads	12
2.1	What is shared, what is private	12
2.2	The pthreads interface	12
2.3	The lifecycle	13
2.4	Passing arguments and results	13
2.5	Thread-local data	13
2.6	Who implements the threads?	14
3	Race conditions	15
3.1	The lost update	15
3.2	Check-then-act	15
3.3	Critical sections and atomicity	16
3.4	Data race vs. race condition	16
3.5	Happens-before	16
3.6	Why testing is not enough	17
4	Locks from the hardware up	18
4.1	What a lock must guarantee	18
4.2	Disabling interrupts	18
4.3	Peterson's algorithm — and why it breaks	18

4.4	Atomic instructions	19
4.5	Spinlocks	19
4.6	Spin or sleep?	19
4.7	Memory ordering in one paragraph	20
5	Condition variables and semaphores	21
5.1	Waiting: spin or sleep	21
5.2	Condition variables	21
5.3	The bounded buffer	22
5.4	Semaphores	22
5.5	Classic problems	23
6	Deadlocks	24
6.1	Definition and the four conditions	24
6.2	Resource-allocation and wait-for graphs	24
6.3	Strategies	25
6.4	Lock ordering in practice	25
6.5	Relatives of deadlock	26
7	System calls and user pointers	27
7.1	User mode and kernel mode	27
7.2	Calling conventions	27
7.3	One system call, end to end	28
7.4	Never trust a user pointer	28
7.5	TOCTOU and the double fetch	29
8	Virtual memory	30
8.1	Why virtual memory	30
8.2	Paging and the x86-64 page tables	30
8.3	The TLB	31
8.4	Page faults	31
8.5	Page replacement	32
9	Scheduling	33
9.1	Bursts and metrics	33
9.2	The classic policies	33
9.3	Multi-level feedback queue	34
9.4	Proportional share and Linux	35
9.5	Spin, yield or sleep?	35
10	Implementing threads	36
10.1	The thread control block and the context	36
10.2	States	36
10.3	Birth: a stack and a trampoline	37
10.4	Death: exit, zombies, join, detach	37

10.5 Blocking primitives	37
10.6 Preemption and interrupt safety	37
10.7 User-level vs. kernel-level, revisited	38
11 SWEB: putting it together	39
11.1 The source tree	39
11.2 From power-on to the shell	39
11.3 Every chapter in the code	40
11.4 What A1 changes	40
11.5 Testing	41
11.6 Pitfalls that cost days	41

Chapter 0

The C toolbox

Operating-systems code is C (or, in SWEB, a careful subset of C++) with no safety net: no garbage collector, no bounds checks, no exceptions. Most bugs you will hunt in A1 are pointer and lifetime bugs that corrupt memory silently and crash much later, somewhere else. This chapter is the minimum toolkit: how to build with the right flags, how to let the compiler’s sanitizers find those bugs for you, and the three C ideas every later chapter relies on — pointers, lifetimes, and function pointers.

0.1 Toolchain: from source to a running program

```
gcc -std=gnu17 -Wall -Wextra -g -O1 -fsanitize=address,undefined prog.c -o prog
```

-Wall -Wextra turn on the warnings that catch real bugs (unused results, sign mix-ups, missing returns). Treat a warning as a bug report you have not read yet. The course modules also turn a few warnings into errors, e.g. **-Werror=implicit-function-declaration**: calling a function without a prototype silently truncates pointers to `int` on 64-bit machines.

-g debug information: line numbers in sanitizer reports and in `gdb`.

-O0/-O1/-O2 optimisation. Race bugs and missing `volatiles` often appear only at **-O2**, because the compiler then keeps values in registers and reorders memory accesses. The modules therefore test both a sanitizer build and an optimised build.

`make` rebuilds only what changed. Every module has a `Makefile` with the targets `run` (examples) and `test` (assignment). `make test IMPL=../../solutions/NN-...` builds the tests against the reference solution instead of your own files.

0.2 Sanitizers: let the compiler find your bugs

A sanitizer is compiler instrumentation plus a run-time library. The instrumented program checks every memory access (or every thread interaction) and prints a report at the *first* violation — not at the crash three functions later.

Sanitizer	Flag	Finds
AddressSanitizer	<code>-fsanitize=address</code>	out-of-bounds accesses (heap, stack, globals), use after free, use after return, double free, leaks
UndefinedBehaviorSanitizer	<code>-fsanitize=undefined</code>	signed overflow, bad shifts, misaligned pointers, NULL dereference, out-of-range enum values
ThreadSanitizer	<code>-fsanitize=thread</code>	data races (chapter 3), lock-order inversions (chapter 6)

Reading a report: the first lines name the *kind* of error (`heap-use-after-free`, `stack-buffer-overflow`, ...) and the access; then come three stack traces — where the bad access happened, where the memory was allocated, and where it was freed. Start at the first frame that is in your code.

ASan and TSan cannot be combined (they instrument memory differently), and ThreadSanitizer fails on recent kernels with high address-space randomisation (“*unexpected memory mapping*”). The modules’ test runner starts TSan binaries under `setarch -R`, which disables randomisation for that one process.

0.3 Pointers

A pointer is an address plus a type. The type says how many bytes `*p` reads and how far `p + 1` moves.

```
int a[4] = {1, 2, 3, 4};
int *p = a;           /* an array decays to a pointer to its first element */
p + 2;                /* address of a[2]: moves by 2 * sizeof(int) bytes */
*(p + 2) == p[2];     /* identical: indexing is pointer arithmetic */
sizeof a;             /* 16: the array; sizeof p would be 8: the pointer */
```

Out-parameters. C returns one value. A function that must produce more writes through pointers the caller passes in — `pthread_create(&tid, ...)` stores the new thread’s id in the caller’s variable, and every system call that reads data does the same with a buffer.

void *. A pointer to “something”. It converts to and from any object pointer without a cast in C, but cannot be dereferenced or used in arithmetic until it is converted back to a real type. Generic code (`qsort`, `pthread_create`, a thread library’s argument) passes `void *` and trusts the caller to know the real type.

Integers as pointers. Kernels constantly convert between pointers and integers: a system call receives user pointers as plain numbers in registers (SWEb: `size_t`, or its `pointer` type). `uintptr_t/size_t` are wide enough; `int` is not. This is also why a *number* chosen by a user must be validated before the kernel treats it as an address (chapter 7).

0.4 Lifetimes: stack, heap, static

Storage	Lives	Typical bug
stack (locals)	until the function returns	returning or storing the address of a local; a thread using a pointer to its creator's local after the creator returned
heap (<code>malloc</code>)	until <code>free</code>	use after free, double free, leak (never freed)
static (globals, <code>static</code>)	the whole program	shared between all threads: needs synchronisation (chapter 3)

The stack is the dangerous one for thread code: a loop that passes `&i` to every new thread passes the *same* address, which keeps changing — and may no longer exist when the thread reads it (module 02, `bug1_loop_index.c` and `bug2_dead_arguments.c`).

```
int *broken(void) {
    int x = 42;
    return &x;      /* x dies here: the caller gets a dangling pointer */
}
```

AddressSanitizer reports this as `stack-use-after-return` (with `detect_stack_use_after_return=1`, which the modules set).

0.5 Function pointers and callbacks

A function's name, used without parentheses, is its address. A function pointer stores such an address and calls through it:

```
int twice(int x) { return 2 * x; }
int (*f)(int) = twice;      /* pointer to function taking int, returning int */
f(21);                      /* 42 */
```

The callback pattern: function + context. A library that calls back into your code cannot know what data your code needs, so it takes a function pointer *and* an opaque `void *` that it hands back unchanged:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine)(void *), void *arg);
```

`start_routine` is “what to run”, `arg` is “with what”. The library never looks inside `arg` — which is also why the SWEB kernel must *not* validate it as a pointer (chapter 7): it may just as well be the number 7 cast to `void *`. Returning a value works the same way backwards: `start_routine` returns a `void *` that `pthread_join` hands to the joiner.

In SWEB

SWEB is written in C++, but kernel C++: no exceptions, no standard library (it brings its own `ustl` containers), fixed-width types (`uint32`, `size_t`, `pointer` for addresses). User programs in `userspace/tests/` are plain C against SWEB's own small `libc` (`userspace/libc`); the `pthread_*` functions there are empty stubs you will fill in.

Key points

- Build with warnings on, and test with sanitizers *and* with optimisation.
- Read a sanitizer report from the top: kind of error, bad access, allocation site, free site.
- Know where every object lives and how long; never hand a pointer to a local to code that outlives the function.
- Callbacks come as *function pointer + opaque context*; the library never interprets the context.

Chapter 1

Processes

A *program* is a file. A *process* is a program being executed: an address space with code and data in it, at least one thread of execution, and a set of resources the kernel keeps on its behalf (open files, the current directory, a process id). This chapter looks at processes from the outside — through the system calls that create, replace, wait for and signal them. Team F implements `fork` and `exec` in SWEB for A1; Team P needs the same picture, because a thread lives *inside* a process.

1.1 What the kernel keeps per process

For every process the kernel has a record — textbooks call it the *process control block* (PCB):

- identity: process id (PID), parent PID, user id;
- the address space: page tables and the list of mapped regions;
- open files: the file-descriptor table;
- the current working directory, signal handlers, exit status;
- one or more threads, each with its own saved registers, stack and scheduling state (chapter 2).

The last point is exactly what SWEB does not have yet: there, a process *is* a thread (`class UserProcess : public Thread`), and the first thing your group does in A1 (“Phase 0”) is to split the two.

1.2 The address space

Every process sees its own, private range of virtual addresses (chapter 8 explains how the hardware makes that possible). On x86-64 Linux the lower half belongs to the process, the upper half to the kernel (figure 1.1).

The heap grows upwards (`malloc` asks the kernel for more with `brk/mmap`), the stack downwards. `/proc/<pid>/maps` shows the real layout of a running process (module 08’s `demand_paging` example prints it).

In SWEB

The boundary is `USER_BREAK` (`arch/x86/64/include/offsets.h`): `0x0000800000000000`. A new SWEB process gets exactly one stack page, the last page below `USER_BREAK`, and its stack pointer starts at `USER_BREAK - 8`. Code and data are *not* loaded up front: each first access page-faults and `Loader::loadPage` copies the page from the binary (chapter 8).

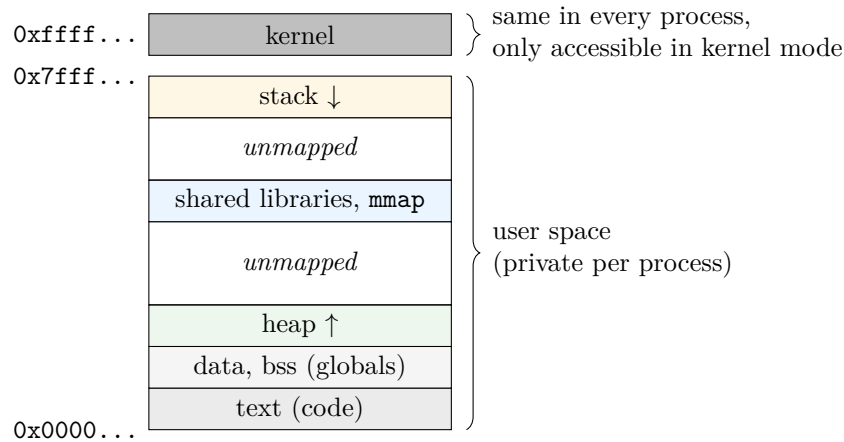


Figure 1.1: The address space of a process on x86-64 Linux (not to scale).

1.3 fork, exec, wait, exit

fork() creates a copy of the calling process: same code, a copy of all memory, copies of the file descriptors (pointing to the same open files). It is called once and *returns twice* — in the parent with the child’s PID, in the child with 0:

```
pid_t pid = fork();
if (pid < 0) { perror("fork"); } /* no child */
else if (pid == 0) { /* child */ }
else { /* parent: pid = the child's PID */ }
```

“Returns twice” is less magic than it sounds: the kernel copies the parent’s saved user registers into the child and then changes one of them — the return-value register (**rax** on x86-64) is 0 in the child. When each of them is next scheduled, both continue at the instruction after the system call. Copying all memory would be slow; real kernels share the pages and copy a page only when one side writes to it (*copy-on-write*, chapter 8).

exec*() replaces the program running in the current process: new code, new data, new stack, same PID, same open file descriptors (unless marked close-on-exec). On success it does not return — the code that called it no longer exists.

exit(status) and wait. A process ends with an exit status; its parent collects it with `wait/waitpid`:

```
int status;
waitpid(pid, &status, 0);
if (WIFEXITED(status)) printf("exit code %d\n", WEXITSTATUS(status));
else if (WIFSIGNALED(status)) printf("killed by signal %d\n", WTERMSIG(status));
```

The shell does exactly this for every command: `fork`, `exec` in the child, `waitpid` in the parent (module 01’s `minish`).

Zombies and orphans. A process that has exited but whose status has not been collected yet is a *zombie*: its memory is gone, but its PCB with the exit status must stay until the parent calls `wait`. A process whose parent dies first is an *orphan*; it is adopted by `init` (PID 1), which reaps it. The same idea returns for threads: a finished, not yet joined thread keeps its return value (chapter 10).

1.4 File descriptors and pipes

A file descriptor is an index into the process's descriptor table; each entry points to an *open file* object in the kernel (position, flags, the file or pipe itself). By convention 0, 1 and 2 are standard input, output and error.

- `fork` copies the table: parent and child share the same open files (and the same file position).
- `dup2(old, new)` makes `new` refer to the same open file as `old` — this is how a shell redirects output.
- `pipe(fds)` creates a kernel buffer with a read end (`fds[0]`) and a write end (`fds[1]`). A reader blocks while the buffer is empty, a writer while it is full.

A reader sees end-of-file only when *every* write end is closed — in every process. A forgotten write end in the parent keeps `wc` waiting forever. Rule: after `fork + dup2`, close every pipe end you do not use, in every process.

1.5 Signals

A signal is an asynchronous notification to a process: `SIGINT` (Ctrl-C), `SIGSEGV` (bad memory access), `SIGCHLD` (a child changed state), `SIGALRM` (a timer expired), `SIGKILL` (cannot be caught). Each has a default action (terminate, core dump, ignore, stop); `sigaction` installs a handler instead.

A handler interrupts the program at an arbitrary instruction — like a hardware interrupt interrupts the kernel. Consequently a handler may only call *async-signal-safe* functions (`write`, `_exit`, ..., but not `printf` or `malloc`, which may hold internal locks at the moment of interruption) and should communicate through a `volatile sig_atomic_t` flag. Chapter 10 uses a timer signal to preempt user-level threads and runs into exactly this rule.

1.6 A trap: buffered output and fork

`printf` does not write immediately: `stdio` collects output in a buffer inside the process and flushes it at a newline (terminal), when the buffer is full (pipe, file) or at exit. `fork` copies that buffer too — output that was “printed” before `fork` but not yet flushed appears *twice*, once from each process. Call `fflush(stdout)` before `fork` (module 01, `stdio_trap.c`; module 08's `cow_fork.c` fell into it while being written).

In SWEB

Baseline SWEB has no `fork` and no `exec`: the shell starts programs with the course's own `createprocess` system call (`common/source/kernel/Syscall.cpp`), which creates a new `UserProcess` for a binary. Implementing `fork` with copy-on-write and `exec` is Team F's part of A1; tearing down every thread of a process on `exit` is the joint work of both teams.

Key points

- A process = address space + resources + threads; the kernel keeps a record (PCB) for each.
- **fork** returns twice because the child gets a copy of the parent's registers with the return value set to 0.
- **exec** replaces the program, keeps the PID and the open files.
- An exited process stays a zombie until its parent waits for it.
- Close unused pipe ends; flush **stdio** before **fork**.
- Signal handlers run at arbitrary points: only async-signal-safe calls inside.

Chapter 2

Threads

A thread is one flow of execution: a program counter, registers and a stack. A process can contain several threads; they run the same program, share its memory and its open files, and are scheduled independently — on a multi-core machine literally at the same time. This is the abstraction you implement in SWEB for A1 (`pthread_create`, `pthread_exit`, `pthread_join`, `pthread_cancel`).

2.1 What is shared, what is private

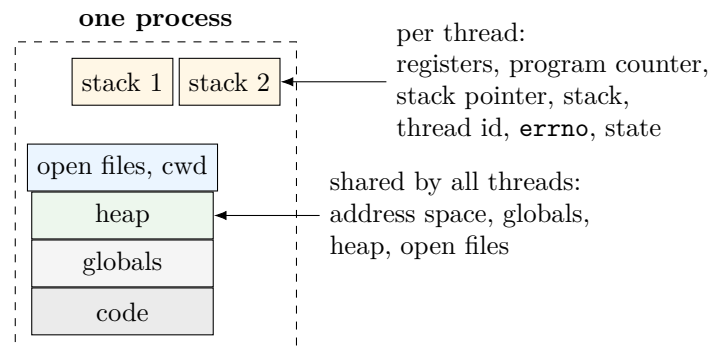


Figure 2.1: Two threads in one process.

Everything reachable through a pointer is shared — including another thread’s stack, if somebody hands out the address of a local variable. “Private” only means “not shared unless you share it”.

Thread vs. process. Creating a thread is much cheaper than creating a process: no new address space, no copied page tables, no copied descriptor table. Switching between threads of the same process does not change the page tables either. The price: no isolation. One thread’s stray write corrupts every other thread, one thread’s crash kills the whole process, and every shared variable needs synchronisation (chapters 3–5).

2.2 The pthreads interface

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine)(void *), void *arg);
void pthread_exit(void *retval); /* never returns */
int pthread_join(pthread_t thread, void **retval);
int pthread_detach(pthread_t thread);
pthread_t pthread_self(void);
```

create starts `start_routine(arg)` in a new thread and stores its id in `*thread`. The new thread may run before or after `pthread_create` returns — never rely on either.

exit ends the calling thread with a return value. Returning from `start_routine` does the same. Returning from `main` (or calling `exit`) ends the *whole process*, all threads included.

join waits until the thread has ended and collects its return value. Afterwards the thread's resources are freed. Each thread can be joined once; joining yourself is an error (`EDEADLK`).

detach “nobody will join this thread”: its resources are freed as soon as it ends. A detached thread cannot be joined.

All functions return 0 or an error number (they do not set `errno`).

2.3 The lifecycle

A thread is created *ready*, becomes *running* when the scheduler picks it, may *block* (waiting for a lock, for I/O, in `join`) and becomes ready again when woken, and finally *terminates*. A terminated thread that is joinable but not yet joined is a *zombie*: its stack can go, but its return value and id must stay until somebody joins it — exactly like a zombie process (chapter 1). Neither joined nor detached means leaked: the zombie stays forever. Chapter 10 builds this state machine.

2.4 Passing arguments and results

Arguments. `arg` is a single `void *`. For more than one value, pass a pointer to a struct — and make sure the struct lives long enough:

```
struct job { int first, count; long result; };
struct job jobs[N];                               /* one per thread, in the caller's frame */
for (int i = 0; i < N; i++) {
    jobs[i] = (struct job){i * chunk, chunk, 0};
    pthread_create(&tid[i], NULL, worker, &jobs[i]); /* not &i ! */
}
for (int i = 0; i < N; i++)
    pthread_join(tid[i], NULL);                    /* jobs[] must outlive the threads */
```

Two classic bugs (module 02, part B): passing `&i` (every thread gets the same, changing address), and passing a pointer to a local of a function that returns before the threads read it.

Results. Either the thread writes into its own struct (as above — no sharing, no lock needed: each thread owns one element), or it returns a pointer. Never return the address of a local; return heap memory, a pointer into the argument, or a small integer cast to `void *`.

Partition, then combine. The simplest correct parallel program splits the data into disjoint parts, lets each thread work only on its part, and combines the partial results *after* `pthread_join`. No shared writes, no locks — and `pthread_join` guarantees that the joiner sees everything the thread wrote (a happens-before edge, chapter 3).

2.5 Thread-local data

Some state must exist once per thread although it looks global. The classic example is `errno`: if thread A's failing call and thread B's failing call wrote the same variable, A could read B's error. C11 has `_Thread_local` (GCC: `__thread`); glibc keeps `errno` in thread-local storage. module 07's mini libc makes its `ml_errno` thread-local for exactly this reason.

2.6 Who implements the threads?

Kernel-level threads (1:1). Every user thread is a thread the kernel knows and schedules.

Blocking in a system call blocks only that thread; threads run in parallel on several cores. Every switch goes through the kernel. Linux (NPTL), Windows — and SWEB after A1.

User-level threads (M:1). A library switches between threads inside one kernel thread (module 10). Switching is a function call, but a blocking system call blocks all threads, and only one core is used.

Hybrid (M:N). M user threads on N kernel threads; complex and mostly abandoned (Go's runtime is a modern variant).

In SWEB

The stubs are in `userspace/libc/src/pthread.c`: `pthread_create` just `return -1`; P1 turns it into a system call that creates a new kernel `Thread` in the calling process (1:1): it gets its own kernel stack (inside the `Thread` object, `kernel_stack_`), its own user stack in the shared address space, and its own saved registers — and starts in user mode at a small libc wrapper that calls `start_routine(arg)` and then `pthread_exit` (chapter 10).

Key points

- Threads share the address space and open files; each has its own registers and stack.
- Returning from the start routine = `pthread_exit`; returning from `main` ends every thread.
- Join or detach every thread; an unjoined finished thread is a zombie.
- Pass each thread its own argument object that outlives it; never `&i`, never a dying local.
- Partition-then-combine needs no locks; `join` makes the results visible.

Chapter 3

Race conditions

Two threads that only read shared data never disturb each other. As soon as one of them writes, the result can depend on the exact order in which the threads' instructions happen to interleave — an order nobody controls. This chapter explains why even `counter++` is not safe, how to reason about interleavings, and which guarantees (“happens before”) you can actually rely on.

3.1 The lost update

`counter++` looks like one step. The CPU executes three:

```
mov  eax, [counter]    ; load
add  eax, 1            ; modify (in a register - private to the thread)
mov  [counter], eax    ; store
```

If thread A is interrupted (or simply runs on another core) between its load and its store, both threads can load the same old value:

Thread A	Thread B	counter
load (reads 5)		5
	load (reads 5)	5
	add, store 6	6
add, store 6		6

Two increments, one effect: a *lost update*. Two threads with three instructions each can interleave in $\binom{6}{3} = 20$ ways; only the 2 in which one thread's three instructions all come before the other's are correct (module 03's `interleave.c` lists all 20). With a million increments per thread, the final count is usually far too small — and different in every run.

3.2 Check-then-act

A race needs no `++`. Any sequence “look at shared state, then act on what you saw” breaks if the state changes in between:

```
if (account->balance >= amount)    /* check */
    account->balance -= amount;      /* act: another thread may have
                                     withdrawn in between */
```

The same pattern appears as “if the file does not exist, create it”, “if the cache has no entry, compute and insert it”, “if the pointer is NULL, allocate”. In a kernel the dangerous variant is

time-of-check to time-of-use (TOCTOU): the kernel checks a value in user memory, then reads it again to use it — and another user thread changed it in between (chapter 7).

3.3 Critical sections and atomicity

A *critical section* is a piece of code that accesses shared state and must not run concurrently with other critical sections on the same state. Making it *atomic* means: every other thread sees either none of its effects or all of them, never an intermediate state. There are three ways to get there (module 03's `three_fixes.c` compares them):

1. **Lock it:** a mutex around the whole check-and-act (chapter 4).
2. **Use an atomic operation:** `atomic_fetch_add` does load–add–store as one indivisible instruction (`lock xadd`). Only works when the whole critical section is a single such operation.
3. **Do not share:** give each thread its own copy and combine afterwards (chapter 2's partition-then-combine).

Invariants are the key to finding critical sections: “the sum of all balances is constant”, “`count` equals the number of elements in the list”. Every code path that temporarily breaks an invariant is a critical section.

3.4 Data race vs. race condition

The two words are often mixed up:

Data race (a precise, mechanical definition): two threads access the same memory location, at least one access is a write, and the two accesses are not ordered by happens-before (no lock, no atomic, no join between them). In C and C++ a data race is *undefined behaviour* — the compiler may assume it never happens. ThreadSanitizer finds exactly these.

Race condition (a semantic property): the program's correctness depends on timing. A program can have a race condition without a data race (every access is locked, but the lock is released between check and act) and a data race without visible harm (a statistics counter that may be a little off — still undefined behaviour).

3.5 Happens-before

Which writes of thread A is thread B guaranteed to see? Only those that *happen before* B's read, where happens-before is built from:

- program order within one thread;
- unlocking a mutex happens before the next lock of the same mutex;
- `pthread_create` happens before the first instruction of the new thread; the thread's last instruction happens before `pthread_join` returns;
- a release store to an atomic happens before an acquire load that reads it (chapter 4);
- transitivity: A before B and B before C gives A before C.

Anything else is unordered, and without an ordering there is no guarantee at all — not even that B ever sees A's write. The compiler may keep a variable in a register for the whole loop:

```
while (!done) {}           /* with a plain `int done`, -O2 may turn this into
                           `if (!done) for (;;)`, - it never re-reads it */
```

and the CPU may make stores visible to other cores in a different order than the program wrote them (chapter 4). `volatile` stops the compiler from caching, but orders nothing between threads; use atomics or locks.

3.6 Why testing is not enough

A race that loses one update in a million runs passes every normal test. It also depends on the machine: more cores, a different load, an optimised build or an extra `printf` change the timing. Hence the modules test twice — under ThreadSanitizer, which reports the data race whenever the unordered accesses *happen*, whether or not they collide in that run, and optimised at full speed, where broken invariants become visible.

In SWEB

SWEB is preemptive: the timer interrupt can switch threads at almost any instruction, even in kernel code (`syscallHandler` enables interrupts before calling `Syscall::syscallException`). After A1, several threads of one process can be in the kernel at the same time, touching the same process structures — the thread list, the address space, the file descriptors. Every such structure needs a lock, and every “check, then act” in your `pthread_join` (“is the thread finished? if not, sleep”) is a critical section. The A1 plan says it bluntly: *racess appear on the tenth run, not the first.*

Key points

- `x++` is load–modify–store; interleavings lose updates.
- Check-then-act is a race even without arithmetic; in the kernel it is TOCTOU.
- Find critical sections through invariants; fix them by locking, atomics, or not sharing.
- Data race = unordered conflicting accesses (undefined behaviour); race condition = timing-dependent correctness.
- Only happens-before edges (locks, create/join, atomics) guarantee visibility; `volatile` is not synchronisation.

Chapter 4

Locks from the hardware up

A lock turns a critical section into something atomic. But how do you build a lock if every load and store can be interleaved with those of other threads? This chapter answers it from the bottom: the requirements, the one-CPU trick (disable interrupts), a classic software algorithm that fails on real hardware, the atomic instructions that make locks possible, and the choice between spinning and sleeping — which is exactly the difference between SWEB’s `SpinLock` and `Mutex`.

4.1 What a lock must guarantee

Mutual exclusion at most one thread in the critical section.

Progress if the lock is free and threads want it, one of them gets it (no deadlock among the contenders).

Bounded waiting a waiting thread gets its turn after a bounded number of others (no starvation) — “fairness”.

and in practice: cheap when uncontended, and not burning CPU time while waiting for long.

4.2 Disabling interrupts

On a single CPU, the only way another thread can run in the middle of your critical section is a timer interrupt that makes the scheduler switch. Turn interrupts off (`cli`), do the work, turn them back on (`sti`) — nothing can interrupt you. This is how kernels protected data before multi-core, and SWEB still uses it for the scheduler’s own data (`Scheduler::schedule` runs with interrupts off). But:

- it is privileged — user code cannot do it;
- it does not stop the *other* cores on a multi-core machine;
- with interrupts off, timer ticks and device interrupts are delayed; the section must be very short, and must not sleep.

4.3 Peterson’s algorithm — and why it breaks

With only loads and stores, two threads can still achieve mutual exclusion on paper:

```
int flag[2], turn;                /* shared */
void lock(int me) {
    int other = 1 - me;
    flag[me] = 1;                 /* I want in          */
    turn = other;                 /* but you go first   */
}
```

```

while (flag[other] && turn == other) {} /* wait while you want in and it's your turn */
}
void unlock(int me) { flag[me] = 0; }

```

It is correct under *sequential consistency* — the assumption that all memory operations happen in one global order that respects each thread’s program order. Real CPUs do not guarantee that. On x86, a store goes into the core’s *store buffer* and becomes visible to the other core later; a subsequent load from a *different* address can overtake it. Both threads store their `flag`, both load the other’s still-old `flag = 0` from memory, and both enter (module 04’s `peterson.c` shows the violation within seconds). A *memory fence* (`mfence`, `atomic_thread_fence(memory_order_seq_cst)`) between the stores and the loads repairs it. The lesson: correct synchronisation needs help from the hardware.

4.4 Atomic instructions

Test-and-set / exchange `old = xchg(&lock, 1)` writes 1 and returns the previous value, atomically. If `old` was 0, you have the lock. (`xchg` with memory is implicitly locked on x86.)

Compare-and-swap `CAS(&x, expected, new)`: if `x == expected`, write `new` — all at once — and report success. The universal building block: any read-modify-write can be written as a CAS loop (`lock cmpxchg`).

Fetch-and-add `old = fetch_add(&x, n)` (`lock xadd`): counters, and the ticket lock below.

C11 exposes them in `<stdatomic.h>`; GCC also has builtins such as `__sync_lock_test_and_set`, which SWEB uses.

4.5 Spinlocks

```

void spin_lock(atomic_int *l) {
    for (;;) {
        if (!atomic_exchange(l, 1)) return; /* test-and-set: got it */
        while (atomic_load(l)) {}           /* test: wait with plain reads */
    }
}
void spin_unlock(atomic_int *l) { atomic_store(l, 0); }

```

Waiting with plain reads (*test-and-test-and-set*) matters: every `xchg` needs exclusive ownership of the cache line, so a crowd of cores hammering `xchg` keeps moving the line between them and slows down even the lock holder. Reads can share the line until the holder’s unlock invalidates it.

A plain spinlock is unfair — whoever happens to win the race gets it. The **ticket lock** is FIFO: `my = fetch_add(&next, 1); while (serving != my) {}`; unlock increments `serving`. Like a deli-counter ticket.

4.6 Spin or sleep?

Spinning wastes the CPU for the whole wait. That is fine when the lock is held for a few instructions and the holder is running on another core. It is disastrous when the holder is *not running* — on a single CPU the spinner burns its entire time slice while the holder cannot make progress. Hence the three options:

1. spin (short critical sections, multi-core, holder never sleeps);
2. spin, but `yield` after a few tries (gives the holder a chance to run);

3. **sleep**: put yourself on a wait queue, block, and let the unlocker wake you. Costs two context switches, wastes no CPU. Linux's `futex` makes the uncontended case a single atomic instruction in user space and enters the kernel only to sleep and wake.

4.7 Memory ordering in one paragraph

Atomic operations also order the *other* memory accesses around them. Lock acquisition needs *acquire* semantics (nothing from the critical section may move before it), unlock needs *release* semantics (nothing may move after it). A release store paired with an acquire load that reads it creates a happens-before edge (chapter 3). C11's default, `memory_order_seq_cst`, is the strongest and simplest; use it unless you know exactly why a weaker order suffices.

In SWEB

`common/source/kernel/`: `SpinLock` tries `ArchThreads::testSetLock` (an `xchg`); if the lock is taken it records itself as waiting and loops *test-and-set + yield* — spinning without yielding would be pointless on SWEB's single CPU. `Mutex` sleeps instead: on contention it enqueues the thread in its waiters list and calls `Scheduler::sleep()`; `release` pops a waiter and wakes it. Both inherit from `Lock`, which keeps the per-thread lists used for deadlock checks (chapter 6). User-space locks in SWEB (`pthread_spin_*`, and mutexes that sleep only through a small system call) are part of P1's elective.

Key points

- Requirements: mutual exclusion, progress, bounded waiting.
- Disabling interrupts works only on one CPU and only in the kernel.
- Peterson needs sequential consistency; store buffers break it without fences.
- Locks are built from atomic read-modify-write instructions (`xchg`, `CAS`, `fetch-and-add`).
- Spin briefly, otherwise sleep; on one CPU never spin without yielding.
- Lock = acquire, unlock = release: that is what makes the data written inside visible to the next holder.

Chapter 5

Condition variables and semaphores

A mutex answers “who may touch the data now?”. Threads also need to *wait for something*: for an element in a queue, for a thread to finish, for the barber to wake up. Waiting correctly — without burning CPU and without missing the moment the condition becomes true — is what condition variables and semaphores are for. Your `pthread_join` in SWEB is one such wait; P1’s elective asks for mutexes, condition variables and semaphores in SWEB’s user space.

5.1 Waiting: spin or sleep

```
while (queue_empty(q)) {}           /* spinning: burns the CPU, and on one CPU
                                     the producer cannot even run */
```

Waiting should *block*: take the thread off the CPU until another thread says “something changed”. Two problems must be solved together: the waiter must not sleep through the signal (the *lost wake-up*), and the condition must be checked and the data accessed under a mutex.

5.2 Condition variables

```
pthread_mutex_lock(&m);
while (!condition)                /* ALWAYS a loop */
    pthread_cond_wait(&cv, &m);    /* atomically: unlock m and sleep;
                                     on wake-up: lock m again */
/* condition holds, and we hold m */
pthread_mutex_unlock(&m);

/* the other side */
pthread_mutex_lock(&m);
condition = true;
pthread_cond_signal(&cv);          /* wake one waiter (broadcast: all) */
pthread_mutex_unlock(&m);
```

A condition variable has no memory: a signal with nobody waiting is lost — by design. The *condition* lives in ordinary shared variables, protected by the mutex; the condition variable only provides “sleep until somebody might have changed it”.

Why wait must unlock atomically. Suppose the waiter unlocked, and only then went to sleep. In between, the signaller could lock, set the condition, signal (nobody waits yet: lost) and unlock — and the waiter sleeps forever. `pthread_cond_wait` makes “unlock and sleep” one step with respect to other threads. module 05’s `lost_wakeup.c` shows the broken and the fixed version.

Why while, not if. Condition variables in pthreads (and practically everywhere) have *Mesa semantics*: a signal only makes the waiter runnable; by the time it has re-acquired the mutex, another thread may have consumed what it was woken for (a *stolen* wake-up). POSIX also allows *spurious* wake-ups without any signal. Rechecking in a loop handles both. (Under the historical *Hoare semantics* the signaller hands the mutex directly to the woken thread, so the condition is guaranteed to hold — simpler to reason about, more expensive to implement.)

Signal or broadcast? `signal` wakes one waiter: correct when every waiter waits for the same thing and any one of them can proceed. When waiters wait for different conditions on the same variable, or when a change may allow several to proceed, use `broadcast` — a lost signal is a hang, an extra wake-up only costs time.

5.3 The bounded buffer

The pattern to memorise (module 05, `bounded_buffer.c`):

```
void put(struct buf *b, int item) {
    pthread_mutex_lock(&b->m);
    while (b->count == CAP)
        pthread_cond_wait(&b->not_full, &b->m);
    b->item[(b->head + b->count++) % CAP] = item;
    pthread_cond_signal(&b->not_empty);
    pthread_mutex_unlock(&b->m);
}

int get(struct buf *b) {
    pthread_mutex_lock(&b->m);
    while (b->count == 0)
        pthread_cond_wait(&b->not_empty, &b->m);
    int item = b->item[b->head];
    b->head = (b->head + 1) % CAP;
    b->count--;
    pthread_cond_signal(&b->not_full);
    pthread_mutex_unlock(&b->m);
    return item;
}
```

5.4 Semaphores

Dijkstra’s semaphore is a counter with two atomic operations: P (`wait`, `sem_wait`): wait until the value is positive, then decrement; V (`signal`, `sem_post`): increment, waking a waiter if any. Unlike a condition variable, a semaphore *remembers*: a `post` with no waiter is kept in the value. Three roles:

Role	Initial	Use
mutual exclusion	1	<code>wait</code> ; critical section; <code>post</code> (a “binary semaphore” — but anyone may <code>post</code> , unlike a mutex)
signalling	0	thread A <code>waits</code> until thread B <code>posts</code> : “B’s event happened”; the order of the two calls does not matter
counting	N	N identical resources (buffer slots, connections)

The bounded buffer with semaphores: `empty = CAP`, `full = 0`, `mutex = 1`; the producer does `wait(empty)`; `wait(mutex)`; `put`; `post(mutex)`; `post(full)`, the consumer the mirror image. The order of the two `wait`s matters: `wait(mutex)` first and then blocking on `empty` while holding it deadlocks the consumer.

A semaphore is easily built from a mutex and a condition variable (module 05, part A) — and conversely; they are equally powerful.

5.5 Classic problems

Readers–writers. Many readers may read at once, a writer needs exclusive access. The simple solution (readers count themselves; the first reader locks writers out, the last one lets them in) lets a steady stream of readers *starve* writers. A writer-preferring lock makes new readers wait as soon as a writer is waiting (module 05, part B); a fair lock queues everybody in arrival order.

Sleeping barber. A barber sleeps when there are no customers; a customer wakes him, or waits in one of N chairs, or leaves when all chairs are taken. It combines signalling (customer wakes barber, barber calls customer) with a counted resource (chairs) and needs care at the edges: a customer arriving just as the barber decides to sleep must not be lost, and shutdown must wake a sleeping barber (module 05, part C).

Monitors. A monitor bundles data, a mutex that every operation holds, and condition variables — the “one object, one lock” discipline as a language feature (Java’s `synchronized` with `wait/notify`). The bounded buffer above is a monitor written by hand.

In SWEB

SWEB’s `Condition` (`common/source/kernel/Condition.cpp`) is a Mesa condition variable bound to a `Mutex`. `wait` puts the thread on the waiters list *while holding the list’s lock*, releases the mutex, and sleeps. The gap between unlocking the list and `Scheduler::sleep()` is closed from the other side: `Scheduler::wake` yields until the thread really is `Sleeping` before it sets it `Running` — no lost wake-up. `Condition::wait` also asserts that the thread holds no other lock while it sleeps. For `pthread_join`: the joiner checks “is the thread finished?” and waits on such a condition, under the lock that the exiting thread takes to set “finished” and signal.

Key points

- Wait by blocking; the condition lives in shared variables under a mutex.
- `cond_wait` unlocks and sleeps atomically — that is what prevents the lost wake-up.
- Always `while (!condition) wait`: Mesa semantics, stolen and spurious wake-ups.
- When unsure, `broadcast`.
- Semaphores remember posts: mutex (1), signal (0), count (N).
- Readers–writers: watch for writer starvation; barber: watch the edges and the shutdown.

Chapter 6

Deadlocks

Locks fix races and create the next problem: threads that wait for each other forever. With two teams adding locks to shared SWEB structures — the thread list, the address space, the scheduler — your group has to agree on how locks are taken. This chapter gives the theory (Coffman’s conditions, graphs, prevention, avoidance, detection) and the practice (lock ordering, validators), plus the relatives of deadlock: livelock, starvation and priority inversion.

6.1 Definition and the four conditions

A set of threads is *deadlocked* if every thread in the set waits for an event that only another thread in the set can cause. The smallest example: thread 1 holds lock A and waits for B, thread 2 holds B and waits for A. A deadlock is possible only if all four *Coffman conditions* hold:

1. **Mutual exclusion:** a resource is held by at most one thread.
2. **Hold and wait:** a thread holds resources while waiting for more.
3. **No preemption:** resources cannot be taken away; the holder releases them voluntarily.
4. **Circular wait:** a cycle $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_1$, each waiting for a resource the next one holds.

Break any one and deadlock becomes impossible.

6.2 Resource-allocation and wait-for graphs

Draw threads as circles and resources as squares; an edge resource $\rightarrow$ thread means “held by”, thread $\rightarrow$ resource means “waits for”. With one instance per resource, a *cycle means deadlock*. Collapsing the resources gives the *wait-for graph*: $T_i \rightarrow T_j$ if T_i waits for something T_j holds.

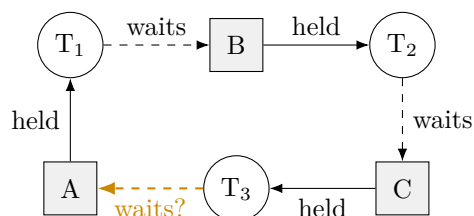


Figure 6.1: T_1 holds A and waits for B, T_2 holds B and waits for C, T_3 holds C. No cycle: T_3 can finish. If T_3 now requests A, the cycle $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$ closes: deadlock.

6.3 Strategies

Prevention — make one condition impossible:

- mutual exclusion: share instead (read-only data, lock-free structures) — only for some resources;
- hold and wait: acquire everything at once, or release everything before waiting (trylock and back off — risks livelock);
- no preemption: take resources away (roll back a transaction) — rarely possible for locks;
- circular wait: **a global lock order**. If every thread takes locks in the same order, no cycle can form. This is the standard technique in kernels.

Avoidance — grant a request only if the system stays in a *safe state*: one from which some order lets every thread finish, even if all of them ask for their declared maximum. The *banker's algorithm* checks this. Example with two resource types A (10 units) and B (5 units):

	Allocation	Max	Need = Max - Alloc
P ₀	(2, 1)	(7, 3)	(5, 2)
P ₁	(3, 1)	(4, 2)	(1, 1)
P ₂	(2, 2)	(5, 4)	(3, 2)

Available = (10, 5) - (7, 4) = (3, 1). Safety check: P₁'s need (1, 1) ≤ (3, 1): it can finish and return its allocation, Work = (6, 2); then P₂ (3, 2) ≤ (6, 2), Work = (8, 4); then P₀ (5, 2). Safe, sequence P₁, P₂, P₀.

- P₀ requests (1, 0): pretend to grant, Available = (2, 1), P₀'s need (4, 2). P₁, then P₂, then P₀ can still finish: safe, grant.
- P₂ requests (1, 1) instead: Available would be (2, 0). No thread's need fits into (2, 0) — unsafe: P₂ must wait, although the units are free.

Avoidance needs to know maximum demands in advance; general-purpose kernels therefore rarely use it, but it is a classic exam topic.

Detection and recovery — let deadlocks happen, find the cycle in the wait-for graph, and break it (kill a thread, roll back). Databases do this. A kernel can at least *report* a deadlock instead of hanging silently.

Ignore it — the “ostrich algorithm”. For locks inside a kernel, the real answer is prevention by lock ordering, backed by tools that check the ordering.

6.4 Lock ordering in practice

- Document a hierarchy: “process lock before thread lock before address-space lock”. Code that holds a lower lock never acquires a higher one.
- Same-level locks (two accounts, two threads): order by a key — the account number, the address of the lock object.
- Never call out into unknown code (callbacks, other subsystems) while holding a lock; you do not know which locks it takes.
- Keep critical sections small, and never sleep while holding a lock somebody else needs to wake you.

Validators. Deadlocks depend on timing, tests rarely hit them. A *lock-order validator* records “A was held while B was acquired” edges during a normal run and reports a cycle as soon as some code path takes B then A — even if the deadlock never happened. Linux calls it *lockdep*; ThreadSanitizer has one built in; module 06, part C builds one.

6.5 Relatives of deadlock

Livelock threads keep running but make no progress — two threads that each trylock, fail, back off and retry in lockstep. Randomised back-off breaks the symmetry.

Starvation the system makes progress, but one thread never gets its turn: an unfair lock, a reader-preferring rwlock, a low priority under strict priority scheduling (chapter 9).

Priority inversion a high-priority thread waits for a lock held by a low-priority thread, while a medium-priority thread (which needs no lock) keeps the low one from running. The high thread effectively waits for the medium one. Mars Pathfinder (1997) reset itself because of this. *Priority inheritance*: the lock holder temporarily runs with the priority of its highest waiter.

In SWEB

Every `Lock` knows its holder (`held_by_`); every `Thread` keeps the locks it holds (`holding_lock_list_`) and the lock it waits for (`lock_waiting_on_`). Before a thread goes to sleep on a lock, `doChecksBeforeWaiting` can walk “waits for lock held by thread that waits for ...” and detect a real deadlock, and it catches recursive locking. `~Thread` asserts that a dying thread holds no locks. Interrupt handlers must not take ordinary locks at all: a handler waiting for a lock held by the thread it interrupted waits forever — a deadlock with a single thread.

Key points

- Deadlock needs all four Coffman conditions; break one.
- Single-instance resources: a cycle in the wait-for graph is a deadlock.
- Kernels prevent deadlock with a documented lock order.
- Banker’s algorithm: grant only if a safe sequence still exists.
- Validators find potential deadlocks from one run; SWEB’s lock lists support similar checks.
- Livelock, starvation and priority inversion are the neighbours to recognise.

Chapter 7

System calls and user pointers

User programs cannot touch devices, other processes or the kernel's memory; the CPU refuses. Everything they need from outside their own address space, they must *ask* for — through a system call, a controlled door into the kernel. Your `pthread_create` is such a door, and every argument that comes through it is chosen by the user program: possibly wrong, possibly malicious.

7.1 User mode and kernel mode

x86 CPUs run in one of four privilege levels (rings); operating systems use two. In *kernel mode* (ring 0) every instruction is allowed. In *user mode* (ring 3) privileged instructions — `cli`, loading `cr3`, I/O ports — raise an exception, and pages whose page-table entries lack the *user* bit are inaccessible (chapter 8). The only ways from user to kernel mode are:

Interrupts from devices (timer, keyboard) — asynchronous, not caused by the running code;

Exceptions caused by the current instruction: page fault, division by zero, general protection fault;

Traps deliberate: the program executes `int n` or `syscall` to request a service.

All three enter the kernel at an address the kernel registered in advance (the *interrupt descriptor table*, or the `syscall` MSRs). The program chooses *which* service — by a number — never *where* the kernel starts executing. An interrupt gate also has a privilege level (DPL): user code may only trigger gates with DPL 3 via `int n`.

7.2 Calling conventions

	Instruction	Number / arguments	Result
Linux x86-64	<code>syscall</code>	<code>rax / rdi, rsi, rdx, r10, r8, r9</code>	<code>rax</code> , errors as <code>-errno</code>
SWEB x86-64	<code>int \$0x80</code>	<code>rax / rbx, rcx, rdx, rsi, rdi</code>	<code>rax</code>

Linux uses `r10` for the fourth argument because the `syscall` instruction itself overwrites `rcx` (return address) and `r11` (flags). SWEB passes the number plus five arguments — exactly enough for `pthread_create`'s four arguments plus the address of a start wrapper.

Errors. The Linux kernel returns `-errno` (a value between `-4095` and `-1`). The libc wrapper turns it into `-1` and stores the positive number in `errno`, a *thread-local* variable (chapter 2). Any

other value, even a “negative” address, is a valid result — hence the test `(unsigned long)r > -4096UL`.

7.3 One system call, end to end

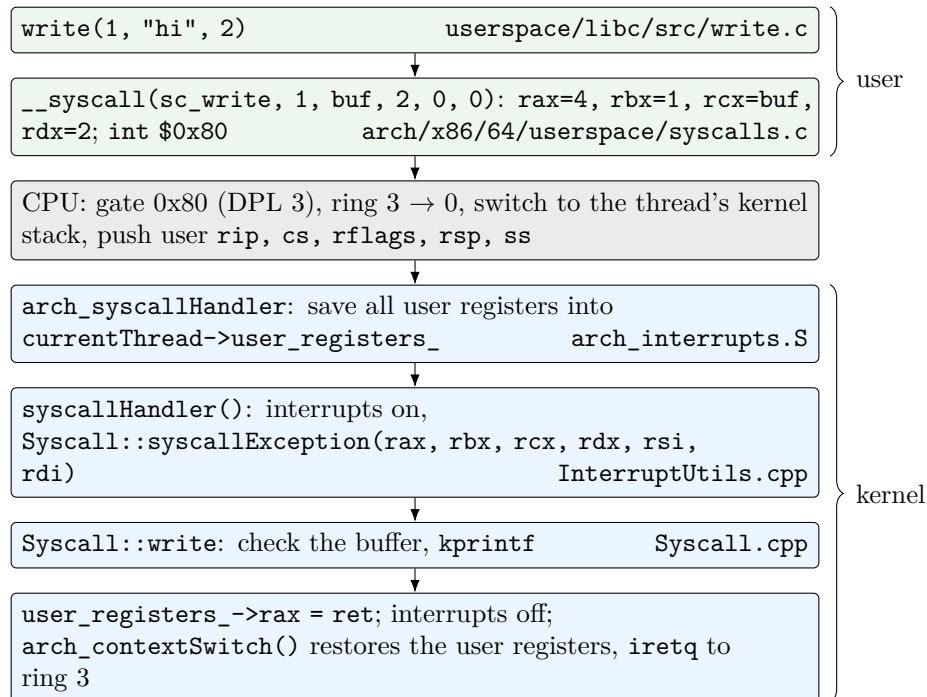


Figure 7.1: The path of `write` through SWEB (x86-64).

Note two details in figure 7.1. The kernel runs the system call on the *thread’s own kernel stack*, so every thread needs one (in SWEB it is part of the `Thread` object). And interrupts are *on* during the call: a system call can be preempted, and several threads can be inside the kernel at once.

7.4 Never trust a user pointer

For `write(fd, buf, len)` the kernel reads `len` bytes at `buf`. If it did so blindly, a program could pass a kernel address and have the kernel copy kernel memory into a file — or an unmapped address and crash the kernel. Every pointer from user space must be checked:

1. **The whole range is user memory.** Not just the start: `buf + len` must also stay below the user/kernel boundary.
2. **The check cannot overflow.** `buf + len > USER_BREAK` fails when `buf + len` wraps around: `buf = 0x1000, len = 0xffffffffffff800` gives `0x800`, “below the break”. Never add two user-controlled numbers; subtract from a constant:

```
if (len > USER_BREAK || buf > USER_BREAK - len) return -EFAULT;
```

3. **The pages exist** (or the kernel handles the fault on a bad user page without dying). Linux copies with `copy_from_user`, which turns a fault into `-EFAULT`; SWEB’s page-fault handler loads valid user pages on demand and ends the process otherwise.
4. **Output buffers are writable.**
5. **Strings** have no known length: copy byte by byte (page by page) into a bounded kernel buffer, checking each page as the copy reaches it — `strlen` on a user pointer *is* the unchecked read.

Not everything that looks like a pointer is one. `pthread_create`'s `arg` is never dereferenced by the kernel — it is passed to the new thread — so it must *not* be checked; `NULL` and `42` are legal. `start_routine` is not dereferenced either; it becomes an instruction pointer and at most needs to lie below `USER_BREAK`.

7.5 TOCTOU and the double fetch

Once a process has several threads, user memory can change *while* the kernel works with it. If the kernel reads a value from user memory to check it and reads it again to use it, another thread can swap it in between:

```
if (!access_ok(iov[i].base, iov[i].len)) return -EFAULT;    /* fetch 1 */
copy_to_device(iov[i].base, iov[i].len);                  /* fetch 2: may differ! */
```

The fix is simple and absolute: copy the user data into kernel memory *once*, then check and use only the copy. (Modern CPUs add SMAP, which makes the kernel fault on any user-memory access outside explicit copy routines.)

In SWEB

Syscall numbers live in `common/include/kernel/syscall-definitions.h`, shared by the kernel and the libc. Adding a system call means: a number, a `case` in `Syscall::syscallException`, a static `Syscall` method, and a libc wrapper calling `__syscall` (module 11, part 5 does it for `gettid`). SWEB's own checks are the hand-written `buffer + size > USER_BREAK` kind — including the overflow hole above. Your `pthread_create` and `pthread_join` receive user pointers (`pthread_t *`, `void **`); check them properly, and check them *before* you create or free anything.

Key points

- The user picks the service number, never the kernel entry point; gates and privilege levels enforce it.
- Linux: `syscall`, args in `rdi rsi rdx r10 r8 r9, -errno`; SWEB: `int $0x80`, number in `rax`, args in `rbx rcx rdx rsi rdi`.
- Every system call runs on the calling thread's kernel stack, with interrupts on.
- Validate the whole user range, overflow-free; copy strings with per-page checks; do not check opaque arguments.
- Fetch user data once; check and use the kernel copy.

Chapter 8

Virtual memory

Every process believes it owns a huge, private address space starting at 0. In reality its pages are scattered over physical memory, many of them are not there at all, and some are shared with other processes. The MMU translates every single memory access, and the kernel keeps the translation tables. This chapter covers the translation (x86-64's four-level page tables, the TLB), the page fault as the kernel's hook into it (demand paging, copy-on-write), and what happens when physical memory runs out (page replacement).

8.1 Why virtual memory

- **Isolation:** a process can only reach the frames its page tables map; the kernel's pages are mapped without the user bit.
- **Relocation:** every program can be linked for the same addresses.
- **Overcommit:** memory is promised (`mmap` of 1 GiB) and delivered page by page on first touch; unused parts cost nothing.
- **Sharing:** the same physical frame can appear in several address spaces (shared libraries, copy-on-write after `fork`).

8.2 Paging and the x86-64 page tables

Memory is divided into 4 KiB *pages* (virtual) and *frames* (physical). A page table maps page numbers to frame numbers. A flat table for a 48-bit address space would have 2^{36} entries of 8 bytes — 512 GiB per process. Instead, x86-64 uses a tree of four levels; each table is one 4 KiB frame with 512 entries, and only the parts of the tree that map something exist.

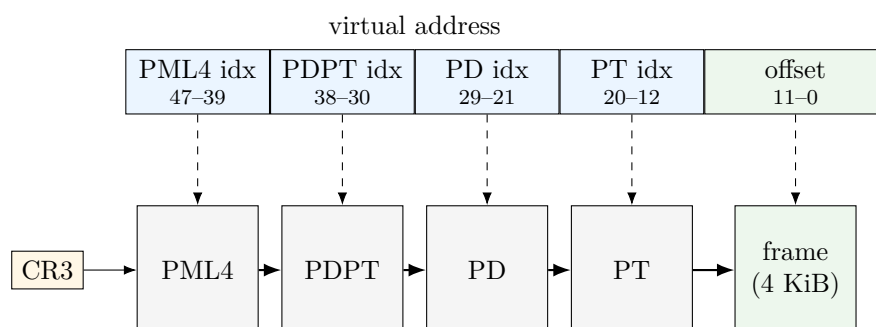


Figure 8.1: A page walk: each 9-bit index selects one of 512 entries; the entry holds the frame number of the next table, and finally of the page.

A page-table entry (64 bits) holds the frame number (bits 12–51) and flags, among them:

Bit	Name	Meaning
0	P (present)	the entry is valid; otherwise any access faults
1	W (writable)	stores allowed
2	U (user)	accessible from ring 3
5	A (accessed)	set by the MMU on every access
6	D (dirty)	set by the MMU on every write
9–11	available	ignored by the MMU, free for the OS (e.g. a COW mark)
63	NX	no instruction fetch

The effective rights are the AND over all four levels: the upper levels are usually created permissive (P, W, U) and the leaf entry decides. Mapping one page into an empty address space costs three new tables (PDPT, PD, PT); unmapping should free tables that became empty.

8.3 The TLB

A page walk costs four extra memory accesses. The *translation lookaside buffer* caches recent translations (page number → frame, rights) and makes almost all accesses hit. The TLB is not coherent with the page tables: when the kernel changes or removes a present entry, it must invalidate the cached copy (`invlpg addr`, or reload `cr3` to drop everything). Forgetting it leaves the old translation usable — after an unmap, a program keeps reading a frame that may already belong to someone else. Entries that were not present are never cached, so *adding* a mapping needs no invalidation. On multi-core machines other CPUs may cache the translation too: a *TLB shutdown* sends them an interrupt to invalidate — expensive, and batched by real kernels.

8.4 Page faults

When the MMU cannot translate (not present) or the rights do not allow the access (write to read-only, user access to kernel page), it raises a page fault: the CPU stores the faulting address (x86: in `cr2`) and an error code (present?, write?, user?) and enters the kernel’s handler. The handler decides:

- legal but not loaded yet: allocate/load the page, map it, return — the CPU *re-executes* the faulting instruction;
- copy-on-write page written: copy it, map the copy writable, return;
- anything else: the program made a mistake — signal `SIGSEGV` (Linux) or end the process (SWEB).

Demand paging. Nothing is loaded before it is used. Code and data are read from the executable on the first access; heap and stack pages are allocated on first touch and **zero-filled** — a frame straight from the free pool may contain another process’s data. On Linux even reading an untouched anonymous page only maps a shared zero page.

Copy-on-write. `fork` shares all frames between parent and child and makes every writable page read-only in *both*, with a software “COW” mark. The first write from either side faults; the handler copies the frame and maps the copy writable. If the frame has only one user left (reference count 1), it simply becomes writable again. Note that the parent’s TLB may still hold writable entries for those pages — they must be invalidated, or the parent’s writes land in the shared frame.

8.5 Page replacement

When no frame is free, a page must be evicted (written to swap if dirty) to make room. Which one?

OPT (Belady) evict the page whose next use lies furthest in the future. Optimal, but needs the future; the yardstick for the others.

FIFO evict the page loaded longest ago. Simple, often bad, and it suffers from *Belady's anomaly*: more frames can cause more faults.

LRU evict the page unused for the longest time. Good, but exact LRU needs a timestamp per access — too expensive in hardware.

Clock (second chance) approximates LRU with the accessed bit: a hand sweeps over the frames; a page with $R = 1$ gets its bit cleared and a second chance, the first page with $R = 0$ is evicted.

Worked example, reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1, 3 frames: FIFO 15 faults, LRU 12, OPT 9, Clock 14. Belady's anomaly with FIFO on 1 2 3 4 1 2 5 1 2 3 4 5: 9 faults with 3 frames, 10 with 4. LRU and OPT are *stack algorithms* — the pages kept with k frames are always a subset of those kept with $k + 1$ — and never show the anomaly. (module 08, part C simulates all four with exact rules.)

Working set and thrashing. The *working set* is the set of pages a process used in the recent past. If the working sets of all running processes do not fit into memory, every process keeps faulting out the pages the others need: the system is busy paging and does no useful work — *thrashing*. The remedy is fewer processes in memory (suspend some), not a faster disk.

In SWEB

arch/x86/64/source/ArchMemory.cpp: `mapPage` walks the four levels and allocates missing tables (P, W, U on the upper levels); `unmapPage` frees tables that became empty; frames come from `PageManager::allocPPN`. The kernel reaches any frame through the identity mapping at `IDENT_MAPPING_START` (`getIdentAddressOfPPN`). Page faults go to `common/source/mm/PageFaultHandler.cpp`; a valid user fault calls `Loader::loadPage`, which only knows the ELF segments — a fault elsewhere, e.g. just below the single stack page, ends the process. For P1 this matters twice: every new thread needs its own user stack somewhere in the shared address space, and two threads of one process can now fault at the same time — `mapPage` has no lock of its own.

Key points

- Four levels of 512-entry tables; the leaf decides the rights, the MMU ANDs all levels.
- Changing or removing a present PTE requires a TLB invalidation.
- Page faults are the kernel's hook: demand paging (zero-filled!), copy-on-write, or kill.
- COW: read-only + mark in both address spaces, copy on first write, no copy for the last user.
- $\text{OPT} \leq \text{LRU}$; FIFO shows Belady's anomaly; Clock approximates LRU with the accessed bit.
- Thrashing: working sets do not fit; reduce the load.

Chapter 9

Scheduling

Whenever more threads are ready than there are CPUs, the scheduler decides who runs — at every timer interrupt, and whenever a thread blocks, wakes up, yields or exits. This chapter covers the metrics, the classic policies you will be asked to simulate by hand in the exam, how real systems combine them, and the question that matters most for your `pthread_join`: should a waiting thread spin, yield, or sleep?

9.1 Bursts and metrics

Threads alternate between *CPU bursts* (computing) and *I/O bursts* (waiting for the disk, the network, the keyboard, a lock). CPU-bound threads have long CPU bursts, I/O-bound (interactive) threads short ones. For a job that arrives at time a , first runs at s and finishes at f with total CPU time b :

Metric	Definition	Who cares
turnaround time	$f - a$	batch jobs
waiting time	$f - a - b$ (time spent ready, not running)	fairness
response time	$s - a$	interactive users
throughput	jobs finished per time unit	the whole system

The goals conflict: short quanta improve response and cost throughput (context switches); favouring short jobs improves the average and can starve long ones.

9.2 The classic policies

Example (arrival, burst, priority): A (0, 5, 2), B (1, 3, 1), C (2, 1, 3), D (4, 2, 1) — module 09, question 1. One box per time unit.

FCFS (first come, first served): run jobs to completion in arrival order. Simple, no starvation, but short jobs wait behind long ones — the *convoy effect*.

A	A	A	A	A	B	B	B	C	D	D
0	1	2	3	4	5	6	7	8	9	10

average waiting 3.75

SJF (shortest job first, non-preemptive): when the CPU becomes free, pick the job with the shortest burst. When all jobs are present from the start, this is provably optimal for the average waiting time (exchange argument: a longer job directly in front of a shorter one — swap them, and the sum of waiting times drops). With staggered arrivals it is only a good heuristic: a short job arriving just after a long one has started still has to wait.

A	A	A	A	A	C	D	D	B	B	B
0	1	2	3	4	5	6	7	8	9	10

average waiting 3.00

SRTF (shortest remaining time first): SJF with preemption — a new job with less remaining work than the running one takes over. Optimal for the average waiting time over all schedules.

A	B	C	B	B	D	D	A	A	A	A
0	1	2	3	4	5	6	7	8	9	10

average waiting 2.00, response 0.25

Both need the length of the next CPU burst, which the OS does not know. It can *predict* it by exponential averaging of the past bursts: $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$.

Round robin (RR): a FIFO ready queue; each job runs for at most one *quantum* q , then goes to the end of the queue. Good response time, fair, no starvation. The quantum is the knob: too small and the CPU is busy switching (a 0.1 ms switch with $q = 1$ ms wastes 9%), too large and RR degenerates into FCFS. Typical values: a few milliseconds. With $q = 2$:

A	A	B	B	C	A	A	D	D	B	A
0	1	2	3	4	5	6	7	8	9	10

average waiting 4.25, response 1.50

A tie that exams love: at $t = 2$, C arrives *and* A's quantum ends. The convention used in **module 09** (and by most textbooks): the newcomer is queued first, the preempted job behind it. State your convention when you answer.

Priority scheduling: always run the most important ready job (here: smaller number = more important; preemptive):

A	B	B	B	D	D	A	A	A	A	C
0	1	2	3	4	5	6	7	8	9	10

average waiting 3.25

Low-priority jobs can *starve* if higher-priority work keeps arriving. Remedy: *aging* — raise the priority of waiting jobs over time. Also watch for priority inversion (chapter 6).

9.3 Multi-level feedback queue

MLFQ learns what kind of job it deals with, without being told:

1. Several queues with decreasing priority; round robin within a queue; higher queues always win, and preempt lower ones.
2. A new job enters the top queue.

3. A job that uses up its time allotment on a level is moved one level down. The allotment is counted *across* I/O — otherwise a job could stay on top by blocking just before its quantum ends.
4. Periodically, all jobs are boosted back to the top: no starvation, and a job that changed its behaviour gets a new chance.

Interactive jobs (short bursts) stay high and get quick responses; CPU-bound jobs sink and share the rest. `module 09`'s bonus implements it with exact rules.

9.4 Proportional share and Linux

Instead of priorities, give each job a *share*: lottery scheduling draws tickets, stride scheduling advances a “pass” value by $1/\text{tickets}$. Linux's scheduler (CFS, since 6.6 EEVDF) is of this kind: each task accumulates *virtual runtime*, weighted by its nice value (nice 0 = weight 1024, each nice step $\approx 1.25\times$ less), and the task with the least virtual runtime runs next. A task that sleeps a lot has little virtual runtime and runs soon after waking — interactive tasks stay responsive without explicit classification (`module 09`'s examples measure both effects).

9.5 Spin, yield or sleep?

A thread that must wait for another thread (a flag, a lock, a join) on a *single* CPU:

spin (`while (!flag) {}`): burns its whole quantum every round — and the flag cannot change meanwhile, because the thread that would set it is not running. Pure waste.

yield (`while (!flag) yield();`): gives the CPU away at once, but stays *runnable*: it is scheduled again and again just to check and yield. Costs a context switch per round, forever.

sleep (block until woken): leaves the ready queue entirely; costs exactly one switch away and one back.

A kernel waits by sleeping, except for very short waits on locks held for a few instructions.

In SWEB

`Scheduler::schedule()` (`common/source/kernel/Scheduler.cpp`) is round robin: it picks the first thread in `threads_` that is `Running` (i.e. schedulable) and rotates the list. The timer interrupt (`irqHandler_0`) calls it every tick, so the quantum is one tick; `Scheduler::yield()` triggers the same code voluntarily with `int $65`. When nothing is runnable, the idle thread runs. `sleep()` sets the state to `Sleeping`, which removes a thread from consideration until `wake()` sets it back. For `pthread_join`: the joiner must *sleep* until the target has exited — a `while (!done) yield();` loop would “work” and waste every round-robin cycle.

Key points

- Metrics: turnaround $f - a$, waiting $f - a - b$, response $s - a$.
- FCFS (convoys), SJF (optimal when all jobs start together), SRTF (optimal), RR (response; quantum trade-off), priority (starvation $\rightarrow$ aging).
- MLFQ: demote on used allotment (across I/O), boost periodically.
- Linux: weighted virtual runtime, nice changes the weight.
- Wait by sleeping; on one CPU, spinning is pointless and yielding in a loop is wasteful.

Chapter 10

Implementing threads

Chapter 2 used threads; this chapter builds them. The mechanics are the same whether the switching happens in a user-level library (`module 10`) or in a kernel (SWEB): a record per thread, a way to save and restore registers, a stack per thread, a state machine, and wait queues. The hard parts are at the edges — how a thread starts, how it ends, who frees its stack, and what happens when an interrupt arrives in the middle of the library’s own bookkeeping.

10.1 The thread control block and the context

For each thread the implementation keeps a *thread control block* (TCB): id, state, saved registers (the *context*), stack, start routine and argument, return value, detached flag, and links for the queues it may sit in.

A context switch saves the running thread’s registers into its TCB and loads the next thread’s registers from its TCB. Loading the stack pointer switches stacks; loading the instruction pointer continues wherever the next thread stopped. From the thread’s point of view, the call that switched away simply returns — later. `swapcontext(&old, &new)` does exactly this in user space; SWEB’s `arch_contextSwitch` restores a `Thread`’s saved `ArchThreadRegisters`.

10.2 States

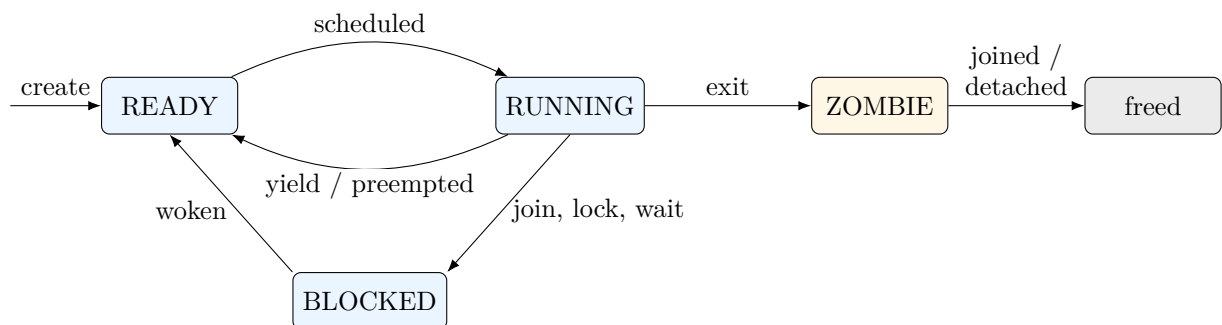


Figure 10.1: Thread states. SWEB’s names: `Running` covers `READY` and `RUNNING`, `Sleeping` is `BLOCKED`, `ToBeDestroyed` is the way out.

A `BLOCKED` thread is in no ready queue; exactly one wait queue (a mutex’s, a condition’s, a joiner slot) knows it, and whoever makes the awaited event happen moves it back to `READY`.

10.3 Birth: a stack and a trampoline

A new thread needs a stack of its own and a first context that “returns” into its code. The first instruction must not be the start routine itself: when `start_routine` returns, it would return to whatever garbage lies on the fresh stack. Instead the thread starts in a small *trampoline*:

```
static void trampoline(void) {
    after_switch();
    pthread_exit(current->start(current->arg));
}
```

/ see "Death" below */*
/ return == exit */*

Stacks should have a *guard page* below them (mapped without any access rights), so that an overflow faults at once instead of silently overwriting the neighbouring stack.

10.4 Death: exit, zombies, join, detach

- **exit**: store the return value, become a ZOMBIE, wake the joiner if one waits, switch away — forever.
- **join**: if the target is already a zombie, collect and free it; otherwise block until its exit wakes you, then collect and free. Errors: no such thread (ESRCH), joining yourself or closing a cycle of joins (EDEADLK), a detached target or a second joiner (EINVAL).
- **detach**: nobody will join; free the thread as soon as it is a zombie (at once, if it already is).

Nobody frees their own stack. A detached thread that exits cannot unmap its stack — it is still running on it; the next push or return would fault. The freeing has to happen *after* the switch, by somebody else: module 10’s library notes the thread in a variable, and the next thread to run frees it right after the switch (in `after_switch`, which the trampoline also calls). SWEB solves the same problem for kernel stacks with a dedicated thread: `Thread::kill` only marks the thread `ToBeDestroyed`, and the cleanup thread `deletes` it later.

10.5 Blocking primitives

A blocking mutex is an owner plus a FIFO of waiting TCBs:

- lock: free → take it; else enqueue yourself, state BLOCKED, switch away;
- unlock: waiters? either *hand the mutex over* to the first one (FIFO, fair, but every contended unlock forces a switch) or free it and wake a waiter who must compete again (*barging*: faster, unfair — SWEB’s `Mutex` does this).

A condition variable is just a wait queue; `wait` enqueues, unlocks the mutex and blocks *without any switch in between*, then re-locks after being woken. Deadlock detection comes almost for free: if the ready queue is empty but some thread is BLOCKED, nothing can ever wake it.

10.6 Preemption and interrupt safety

Without preemption, a thread runs until it calls into the library; atomicity inside the library is automatic. With a timer (module 10’s bonus uses SIGVTALRM, the kernel uses the timer interrupt), a switch can happen *anywhere* — also in the middle of `q_push` or between “mutex is free” and “I am the owner”. The library’s own data then needs protection: block the signal (user space) or disable interrupts (kernel, one CPU) around every operation on it, exactly as SWEB’s scheduler runs with interrupts off. On several CPUs this is not enough; a spinlock is needed as well.

And code that can be interrupted at any instruction must not be re-entered through non-reentrant functions: a thread preempted inside `malloc` holds `malloc`'s lock; another thread on the same kernel thread calling `malloc` deadlocks. SWEB's version of the rule: no `new/delete` with interrupts off or in interrupt handlers.

10.7 User-level vs. kernel-level, revisited

	User-level (M:1)	Kernel-level (1:1)
switch cost	function call + registers (module 10: $\approx 0.4 \mu\text{s}$, mostly glibc's signal-mask syscall)	kernel entry, scheduler, exit ($\approx 3 \mu\text{s}$ in module 10's measurement)
blocking syscall	blocks <i>all</i> threads	blocks only the caller
multiple cores	no	yes
per thread	TCB, user stack	kernel stack, user stack, two register sets, kernel TCB

In SWEB

After A1 every pthread is a kernel `Thread` (1:1). The pieces map one to one:

- TCB = `Thread`; its `kernel_stack_` is 8 KiB inside the object; `user_registers_` and `kernel_registers_` are the two contexts (user mode, and the kernel side of a system call or interrupt).
- Birth: `ArchThreads::createUserRegisters(regs, entry, user_stack_top, kernel_stack);` `entry` = a libc wrapper that calls `start_routine(arg)` and then `pthread_exit`; the two values reach it in `rdi/rsi`. `setAddressSpace` gives the thread the process's page tables.
- The user stack: a new region in the shared address space, below the first thread's stack page, with a guard gap (chapter 8).
- Death and reaping: `kill` $\rightarrow$ `ToBeDestroyed` $\rightarrow$ cleanup thread. Join data (return value, finished) must survive the `Thread` object — keep it in the process.
- Waiting: `Condition/Mutex` (chapter 5), never a yield loop (chapter 9).

Key points

- A context switch = save registers into one TCB, load them from another; the stack pointer switches the stack.
- Start through a trampoline so that returning means exiting; guard pages catch stack overflows.
- Zombies keep the return value until join; detached threads are freed at exit — by somebody else, after the switch.
- Blocking = a wait queue + state `BLOCKED` + switch; hand-off vs. barging.
- Preemption makes the library's own data shared: disable interrupts / block the signal; no non-reentrant calls.

Chapter 11

SWEB: putting it together

SWEB is the course’s teaching kernel: small enough to read, real enough to boot on x86-64 hardware (in QEMU). This last chapter is a map — where things are, how the machine gets from power-on to the shell, where each idea of chapters 1–10 lives in the code — and a guide to the A1 work: what changes, and how to test it so that it survives the tenth run. **module 11’s** worksheet walks through all of it with questions.

11.1 The source tree

Path	Contents
<code>common/source/kernel/</code>	<code>main.cpp</code> (startup), <code>Thread</code> , <code>UserProcess</code> , <code>Scheduler</code> , <code>Syscall</code> , <code>Loader</code> , <code>ProcessRegistry</code> , locks (<code>Lock</code> , <code>Mutex</code> , <code>SpinLock</code> , <code>Condition</code>), <code>CleanupThread</code> , <code>IdleThread</code>
<code>common/source/mm/</code>	<code>PageManager</code> (frames), <code>KernelMemoryManager</code> (kernel heap), <code>PageFaultHandler</code>
<code>common/source/fs/</code> , <code>console/</code>	VFS, minix file system, terminal, <code>kprintf</code> and <code>debug()</code>
<code>common/include/kernel/</code>	headers, <code>syscall-definitions.h</code> (numbers, shared with the libc), <code>user_progs.h</code>
<code>common/include/console/debug.h</code>	debug categories (switch output on/off per category)
<code>arch/x86/64/</code>	page tables (<code>ArchMemory</code>), registers and context switch (<code>ArchThreads</code> , <code>arch_interrupts.S</code>), the IDT and all interrupt handlers (<code>InterruptUtils.cpp</code>), <code>offsets.h</code> (<code>USER_BREAK</code>)
<code>userspace/libc/</code>	SWEB’s own small C library: <code>write.c</code> , <code>pthread.c</code> (stubs), <code>nonstd.c</code> (<code>_start</code>)
<code>userspace/tests/</code>	user programs; every <code>x.c</code> becomes <code>x.sweb</code> on the disk image

11.2 From power-on to the shell

1. GRUB loads the kernel (the menu has no timeout — press Enter); early assembly sets up paging and jumps to `startup()`.

2. `startup()` creates the console, the scheduler (with the cleanup and idle threads), mounts the root file system and adds the `ProcessRegistry` thread; then it enables interrupts and yields — from now on the scheduler runs everything.
3. `ProcessRegistry::Run` mounts the user-program partition at `/usr` and creates a `UserProcess` for each entry of `user_progs` — by default only `/usr/shell.sweb`.
4. The `UserProcess` constructor opens the binary, creates a `Loader` (reads the ELF headers), maps one stack page below `USER_BREAK`, sets the user registers (`rip = ELF entry = _start`, `rsp = USER_BREAK - 8`) and the address space; the scheduler switches to it in user mode.
5. Every first touch of code or data page-faults and is loaded by `Loader::loadPage`. The shell reads commands with `sc_read` and starts programs with `createprocess`.

11.3 Every chapter in the code

Chapter	Where in SWEB
1 processes	<code>UserProcess</code> (= a thread today), <code>ProcessRegistry</code> , <code>createprocess</code> ; no <code>fork/exec</code> yet (Team F)
2 threads	<code>pthread.c</code> stubs; <code>Thread</code> with <code>kernel_stack_</code> and two register sets
3 races	a preemptive kernel: interrupts are on during system calls
4 locks	<code>SpinLock</code> (test-and-set + yield), <code>Mutex</code> (sleeps), <code>ArchThreads::testSetLock</code> , interrupts off in <code>schedule()</code>
5 condvars	<code>Condition</code> ; <code>Scheduler::sleep/wake</code> with the wait-until-sleeping loop
6 deadlocks	<code>holding_lock_list_</code> , <code>lock_waiting_on_</code> , <code>doChecksBeforeWaiting</code> ; no locks in interrupt handlers
7 syscalls	<code>int \$0x80</code> , <code>arch_syscallHandler</code> , <code>Syscall::syscallException</code> , <code>buffer + size > USER_BREAK</code> checks
8 memory	<code>ArchMemory::mapPage/unmapPage</code> , <code>PageManager</code> , <code>PageFaultHandler</code> , <code>Loader::loadPage</code> , identity mapping
9 scheduling	round robin in <code>Scheduler::schedule</code> , one tick per quantum, <code>yield = int \$65</code>
10 threads	<code>Thread</code> states, <code>kill</code> → <code>ToBeDestroyed</code> → cleanup thread

11.4 What A1 changes

Phase 0 (all four, 1–4 Oct): split process from thread. A process owns the address space (`Loader/ArchMemory`), the file descriptors and the PID; a thread owns its stacks, registers, state and TID. Consequences to design together: how a thread finds its process (a pointer; keeping `Thread::loader_` as a non-owning shortcut saves many edits), who destroys the process and when (after its *last* thread, in the cleanup thread — never on a stack or in an address space that is about to disappear), how ids are handed out (today every `tid_` is 0), and what `ProcessRegistry` counts.

Team P. `pthread_create` (P1): the system call, argument checks, a new thread with its own user stack and a start wrapper. `pthread_exit` (P2): the return value survives the thread, the stack is freed, the process keeps running. `pthread_join` (P1): block until the target has finished, receive exactly the stored value; a detached thread cannot be joined. `pthread_cancel` (P2): end the target at a defined cancellation point.

Team F. `fork` with copy-on-write, `exec` — and, together with Team P, their behaviour in multi-threaded processes (`fork` duplicates only the calling thread; `exit` tears down every thread).

11.5 Testing

The course grades with your own tests and with the Tacos test system; neither replaces the other. Practical rules:

- Write the test program first (`userspace/tests/`); re-run CMake after adding one, and build twice if the new program is missing from the image.
- Test the edges, not only the happy path: bad user pointers, joining yourself or twice, main exiting first, a thread creating threads, many threads, create/join in a loop (leaks).
- Run every test many times: module 11's `swab_run.py -n 10` boots SWEB headless ten times and reports hangs (exit 1) and kernel panics (exit 2). Races appear on the tenth run.
- Use the debug categories (`debug.h`) instead of scattering `kprintf`; switch noisy ones off. For hard cases: `make qemuadb + make runadb`.
- Assertions are cheap documentation: `assert(lock.isHeldBy(currentThread))` at the top of a function that needs the lock.

11.6 Pitfalls that cost days

- `new/delete` with interrupts off or in a handler: the kernel-heap lock may be held by the interrupted thread.
- Sleeping while holding a lock the waker needs; `Condition::wait` asserts against extra held locks for a reason.
- Deleting a thread (or an address space) that is still running or still in use — defer to the cleanup thread.
- Trusting user pointers, or checking them with an overflowing sum.
- Busy-waiting (`while (!done) yield();`) instead of sleeping on a condition.
- Two threads faulting on the same missing page table at once: `mapPage` needs a lock once a process has several threads.
- Checking once, using later (TOCTOU) — in user memory and in kernel state: “is the thread still alive?” must be answered and acted on under the same lock.

Key points

- Know the map: `common/source/kernel`, `mm`, `arch/x86/64`, `userspace/libc`, `userspace/tests`.
- Boot path: `startup` → `ProcessRegistry` → `UserProcess` → lazy loading by page faults.
- A1 starts with the process/thread split — agree on ownership, lifetime, ids and locking as a group.
- Test the edges, many times, headless; read the debug output.